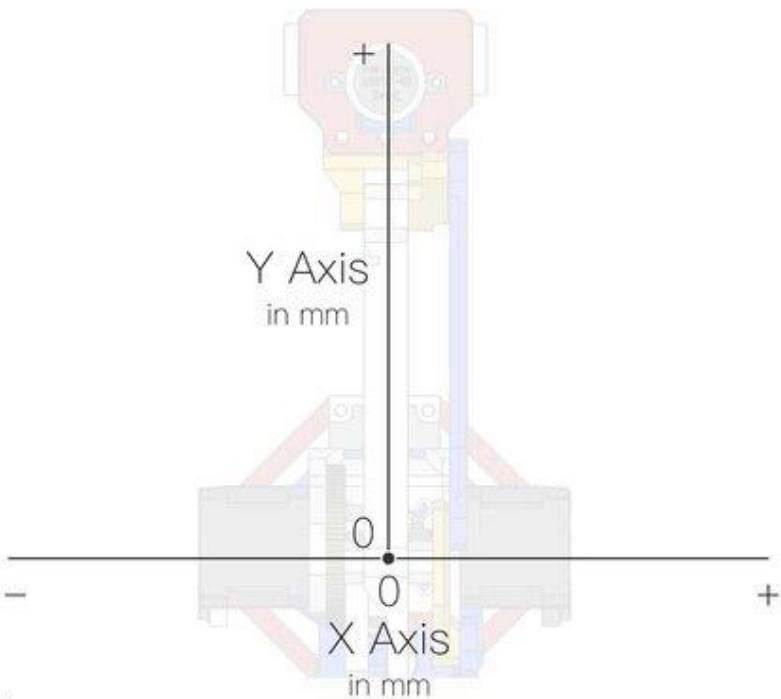


01 – Firmware

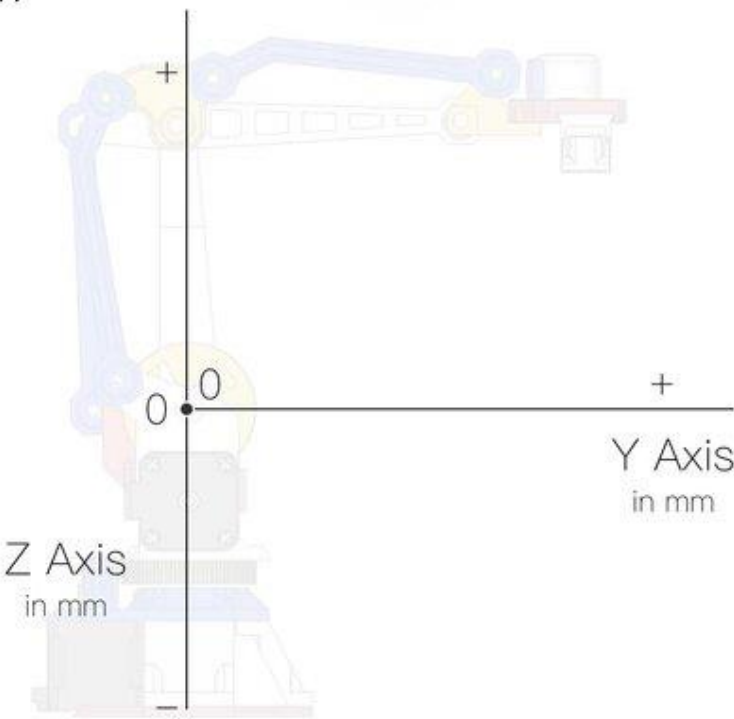
Cartesian Guide

each axis has ranges of coordinates which robot cannot reach, either too far or in conjunction with itself

Top View



Side View

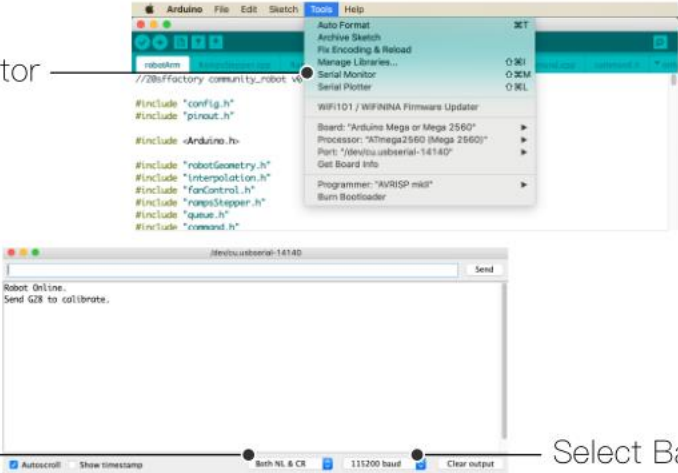


02 – Firmware

Command Guide (Arduino IDE)

Arduino IDE Serial Monitor can perform basic single command serial communications.
Suitable for quick testing after firmware upload is completed.

Open Serial Monitor



Select Line Ending

Both NL & CR

Select Baud Rate

115200

Calibrate Robot

g28

Robot Online.
Send G28 to calibrate.

With Endstops:

Send G28

Without Endstops:

Move Arms into Endstop Positions

Rotate Body to Center Position

Send G28

Command List

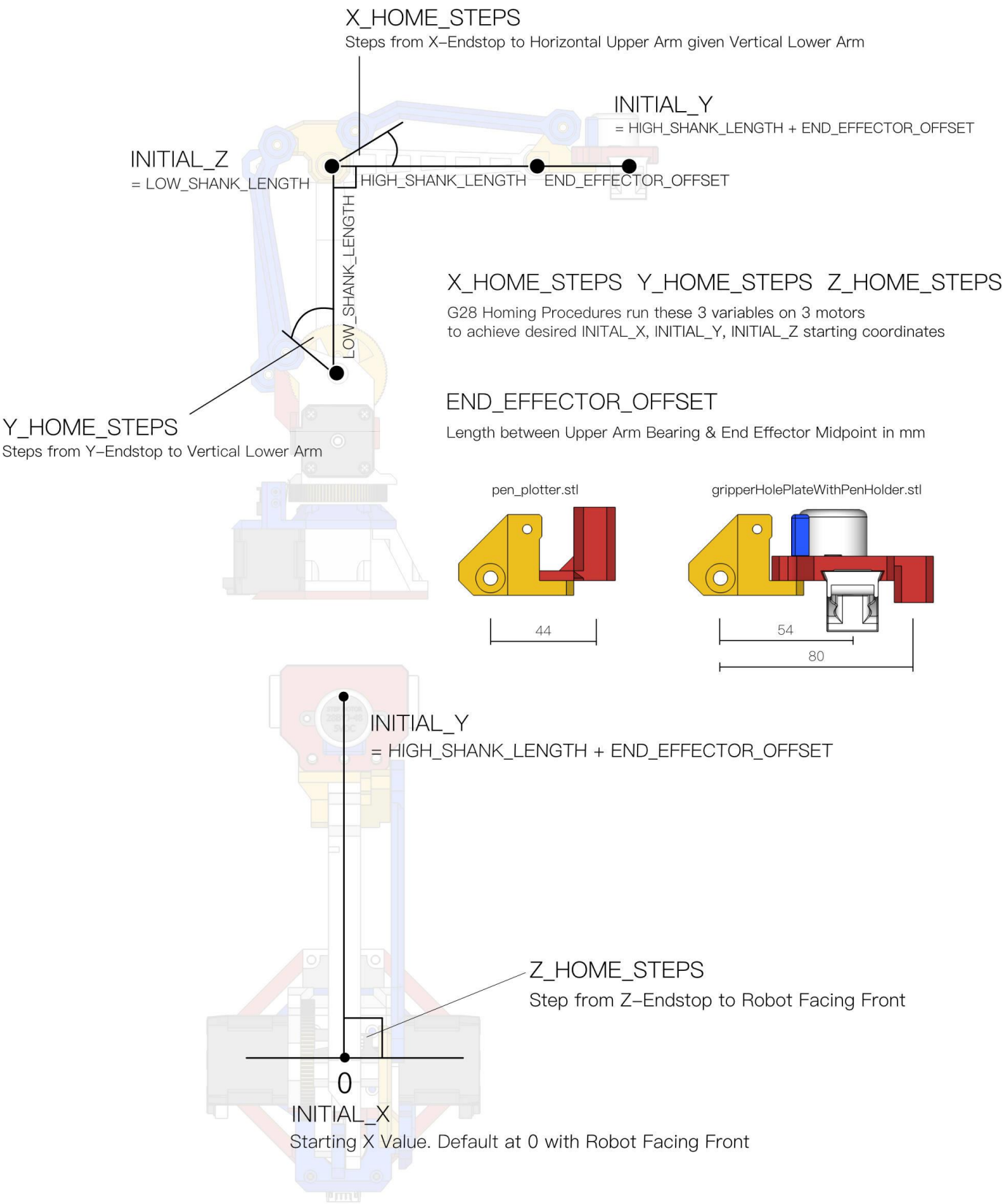
See Cartesian Guide for Coordinate Reference

Linear Move	G0 X<x_mm> Y<y_mm> Z<z_mm> F<mm/s> eg: G0X120Z-10F30	Pump On/Off	M1 / M2
Dwell	G4 S<sec> (waits for <sec> seconds) eg: G4S10	Gripper On/Off	M3 / M5
Homing	G28 (home all motors if homing enabled)	Laser On/Off	M6 / M7
Absolute Mode	G90 (moves in absolute coordinate)	Steppers Enable/Disable	M17 / M18
Relative Mode	G91 (moves in increment value)	Fan Enable/Disable	M106 / M107
Set Position	G92 X<x_mm> Y<y_mm> Z<z_mm> E<mm/s> eg: G92 Z0 (sets current z value to 0) eg: G92 (resets all offset values)	Report Coordinates	M114
Set Speed Curve	M205 S[SPEED_PROFILE] eg: M205 S0 (sets flat speed curve) eg: M206 S2 (sets cosin speed curve)	Report Endstop State	M119

03 – Firmware

Configuration Guide

For parameters not covered in this guide, refer to comments within 'config.h'



04 – Firmware

Homing Calibration Guide

For Robot Arm with All Endstops Installed



G28 Expected Behaviour:

1. Lower Shank swings back to hit Y endstop
2. Lower Shank moves forward by `Y_HOME_STEPS`
3. Upper Shank swings back to hit X endstop
4. Upper Shank moves down by `X_HOME_STEPS`
5. Main Body turns clockwise to hit Z endstop
6. Main Body turns counter-clockwise by `Z_HOME_STEPS`
7. (If RAIL is true) Rail Module moves to hit endstop

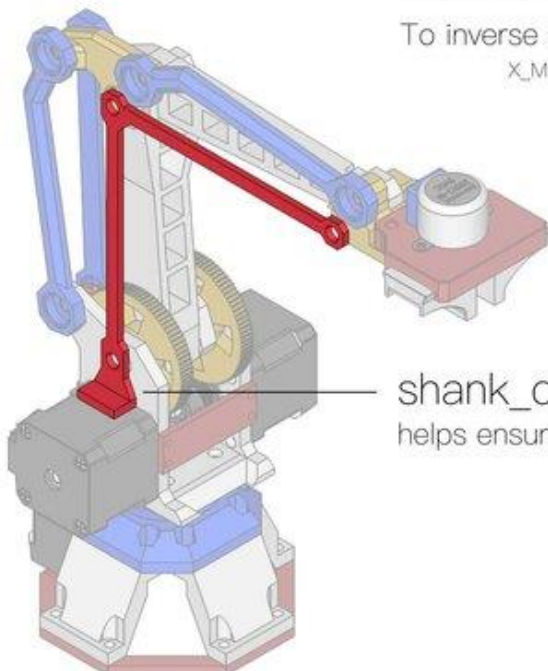
After G28, Robot shanks form right angle facing forward

Endstop Output Values:

Use 'M119' Command to check if endstops are functioning

To inverse activation value, adjust in config.h:

`X_MIN_INPUT, Y_MIN_INPUT, Z_MIN_INPUT`



`shank_calibrator.stl`

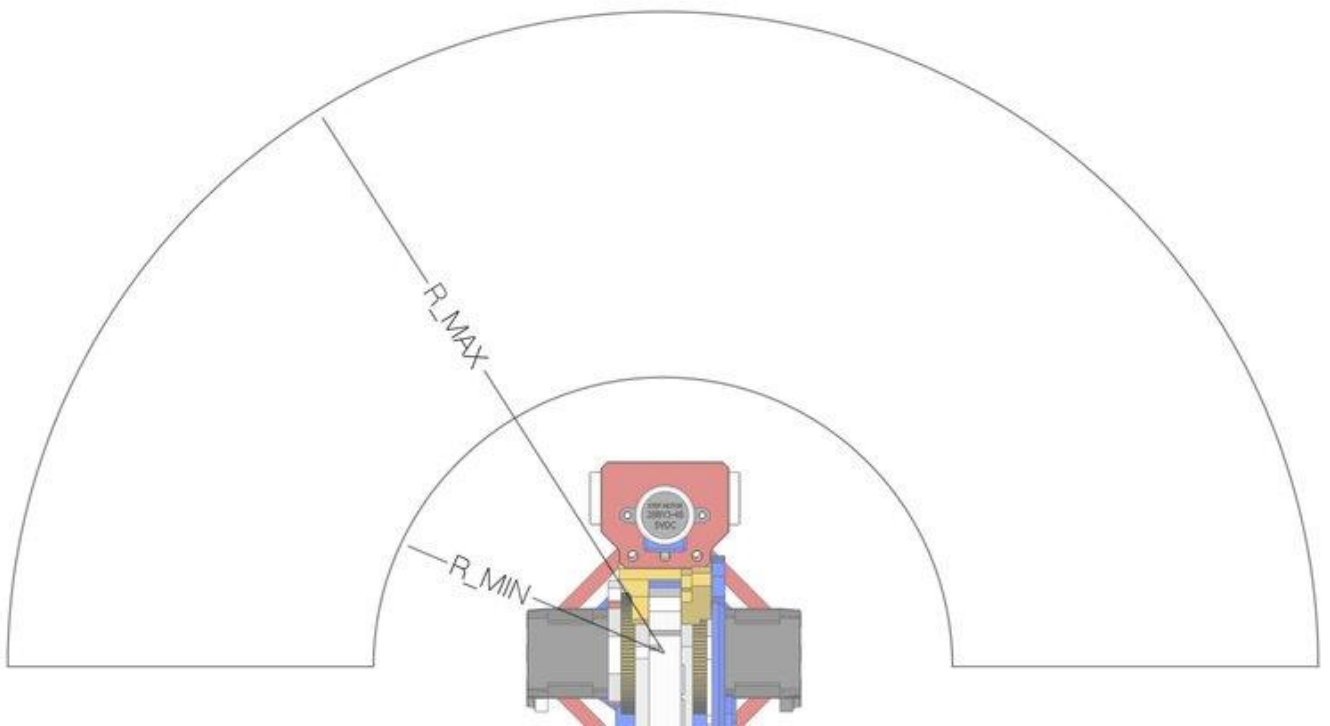
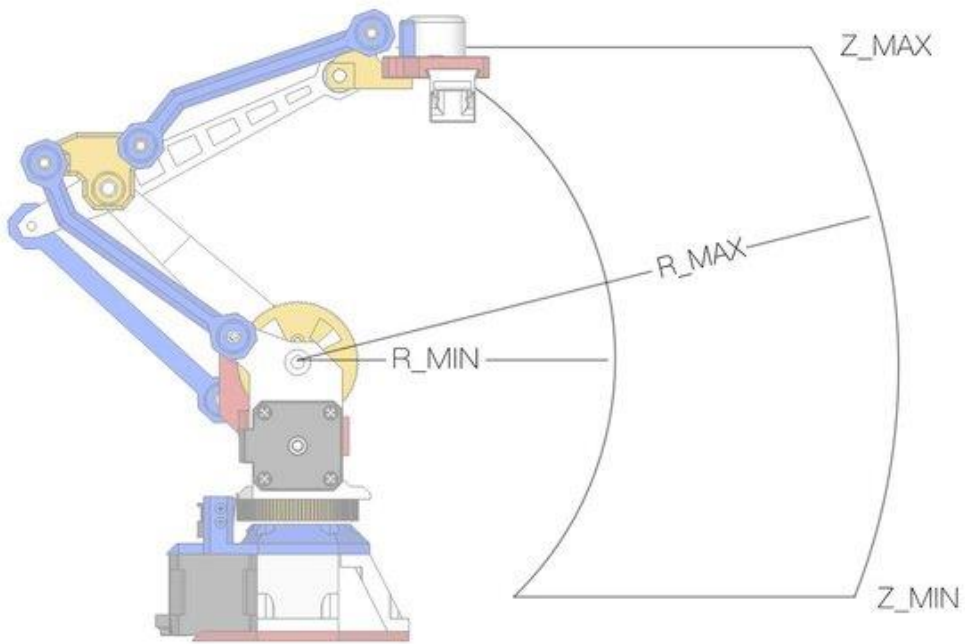
helps ensure lower & upper shank form right angle

05 – Firmware

Operation Range Guide

Z_MAX, Z_MIN, R_MAX, R_MIN Values Located in "config.h"

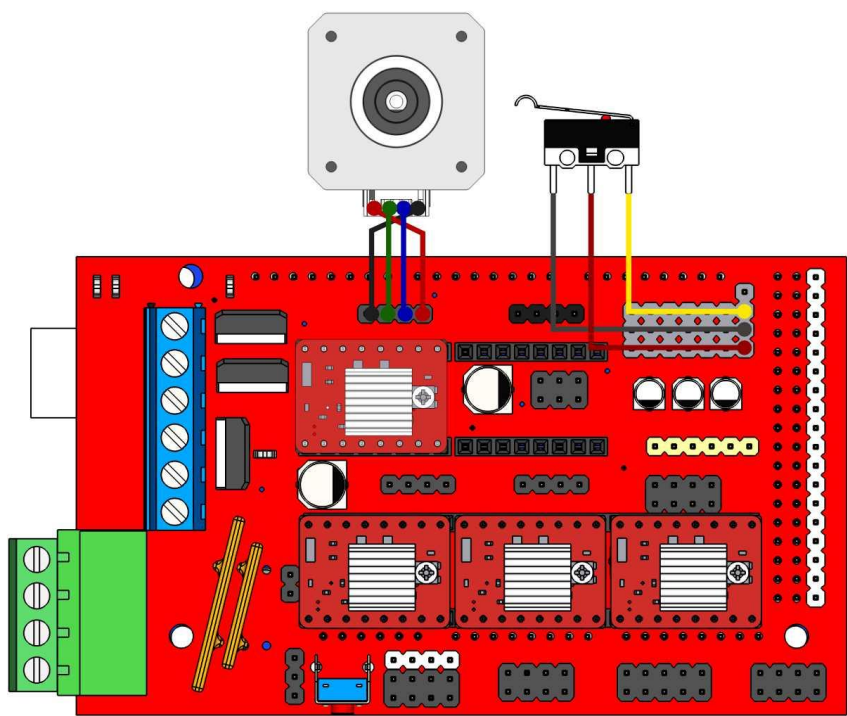
Robot moves beyond range will be terminated



06 – Firmware

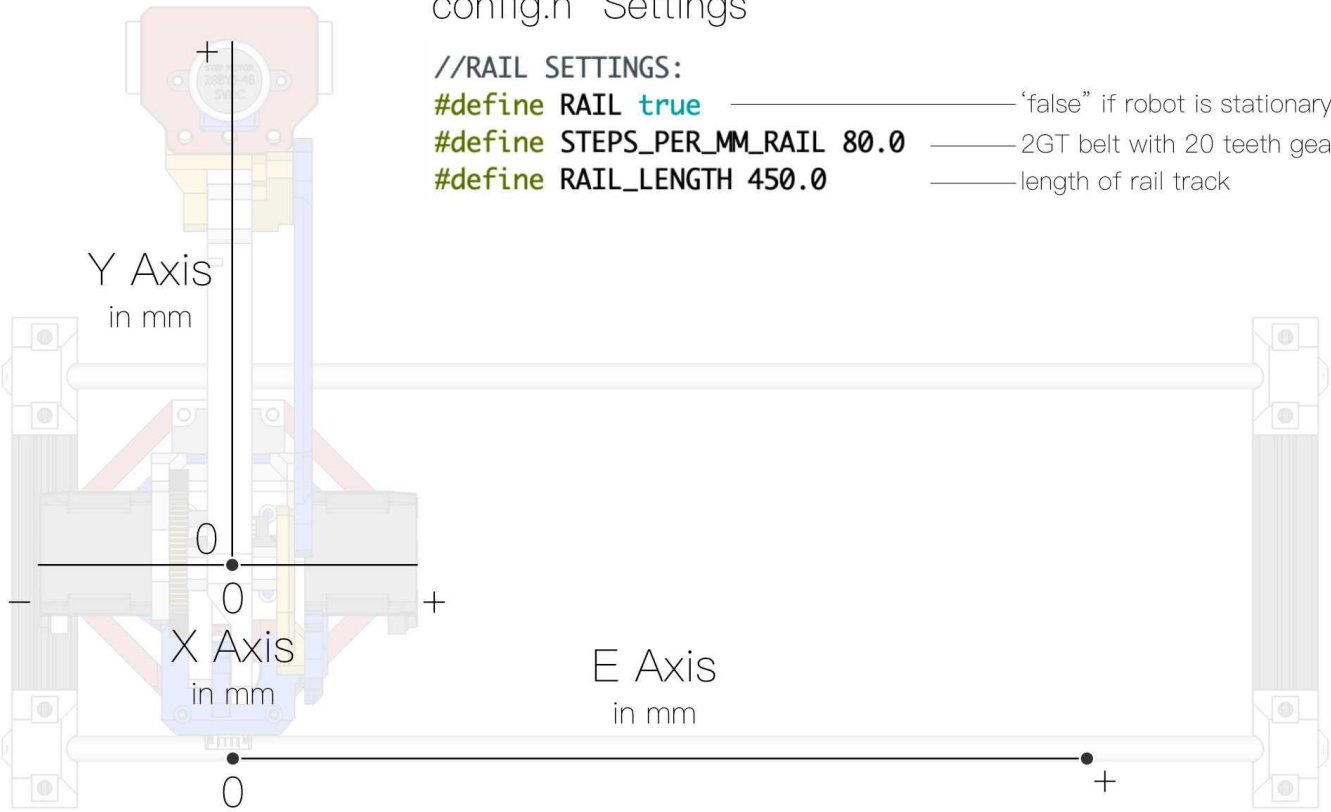
Rail Option Guide

firmware can power an additional E axis on rail
gcode command structure is identical.
(eg. g0x120e120f50)



“config.h” Settings

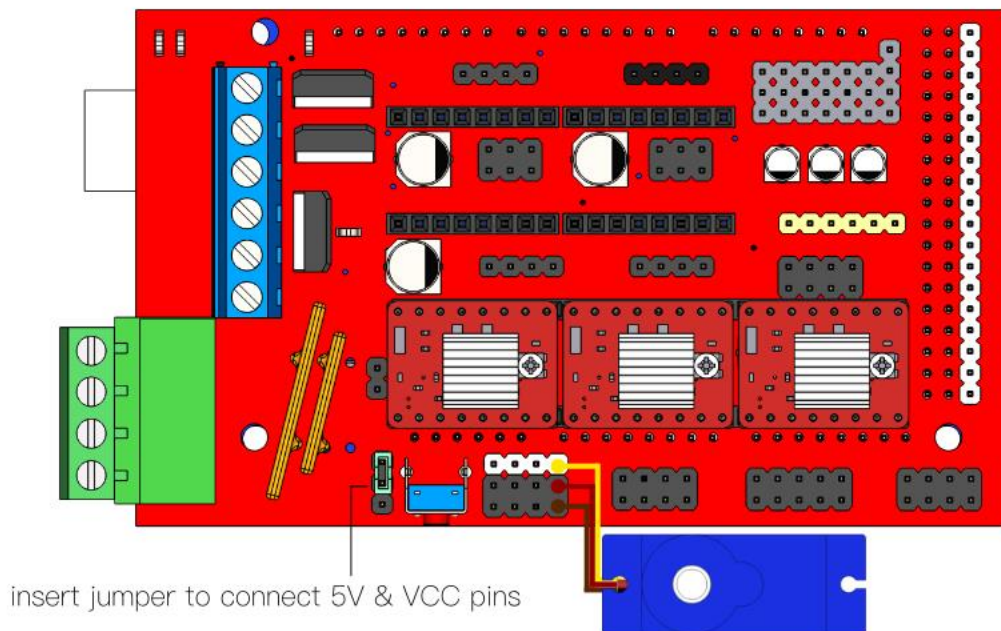
```
//RAIL SETTINGS:  
#define RAIL true //“false” if robot is stationary  
#define STEPS_PER_MM_RAIL 80.0 //2GT belt with 20 teeth gear  
#define RAIL_LENGTH 450.0 //length of rail track
```



07 – Firmware

Servo Gripper Guide

Firmware supports servo driven gripper



“config.h” Settings

```
//GRIPPER SETTINGS
#define GRIPPER 1 //GRIPPER MOTOR IN USE
// 0: 28BYJ-48 MICRO STEPPER MOTOR
// 1: 9G SERVO OR MG996 SERVO EQUIVALENT
//28BYJ GRIPPER SETTINGS
#define BYJ_GRIP_STEPS 1200 //FTOBLER: 1200
//SERVO GRIPPER SETTINGS
#define SERVO_GRIP_DEGREE 90.0 ——— Degree Move of Command ‘M3’
#define SERVO_UNGRIP_DEGREE 0.0 ——— Degree Move of Command ‘M5’
```

Set GRIPPER as 1 to use Servo Gripper

Servo Single Move Logic

- 1 motor.attach(pin_number)
- 2 motor.write(degree)
- 3 delay(300) ——— delay by milliseconds for servo to complete move
- 4 motor.detach() ——— servo is detached after move to avoid overheat or jittering

to amend servo move logic, rewrite servo_gripper.cpp cmdOn() & cmdOff() functions accordingly

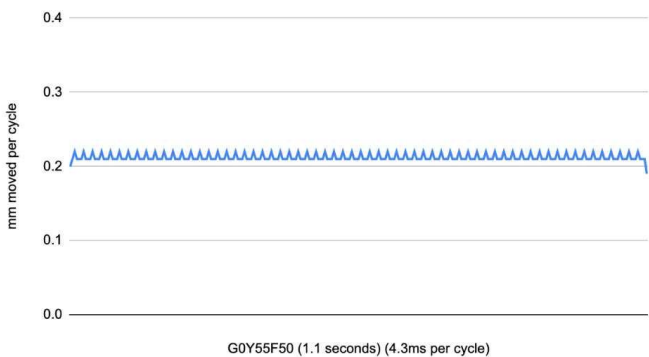
08 – Firmware

Speed Curve Guide

Change SPEED_PROFILE in “config.h” to achieved desired curves

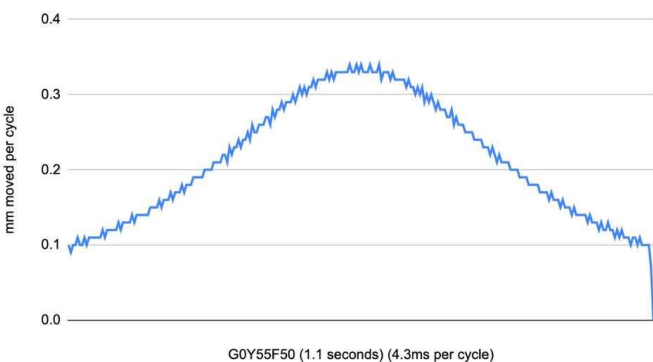
#define SPEED_PROFILE 0

Flat Speed Curve
suitable for constant speed per movement



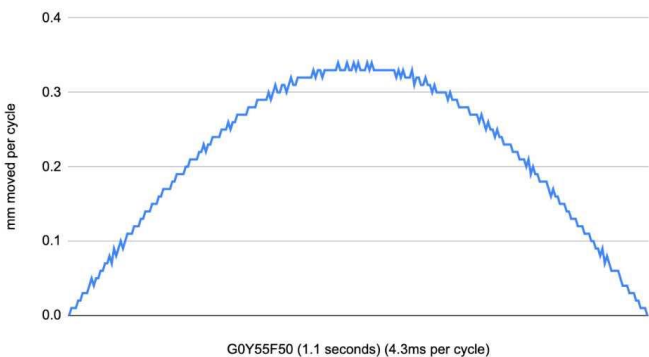
#define SPEED_PROFILE 1

ArcTan Approx



#define SPEED_PROFILE 2

Cosin Approx
Accelerate speed from 0 & Decelerate to 0 per movement



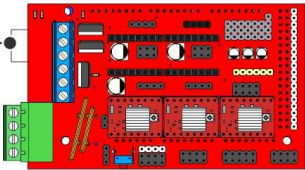
09 – Firmware

Upload Guide

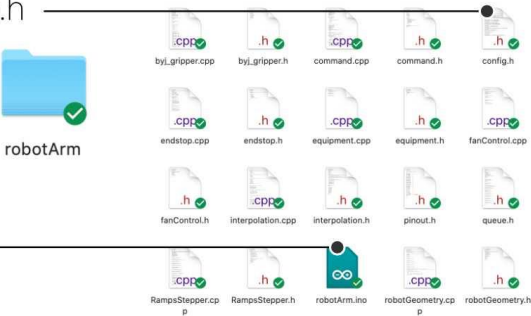
Download Arduino IDE
www.arduino.cc/en/Main/Software



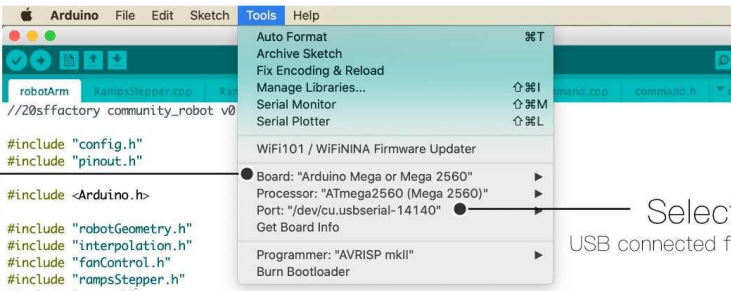
Connect USB Cable



Adjust Custom Parameters in config.h
See “Configuration Guide”



Launch robotArm.ino



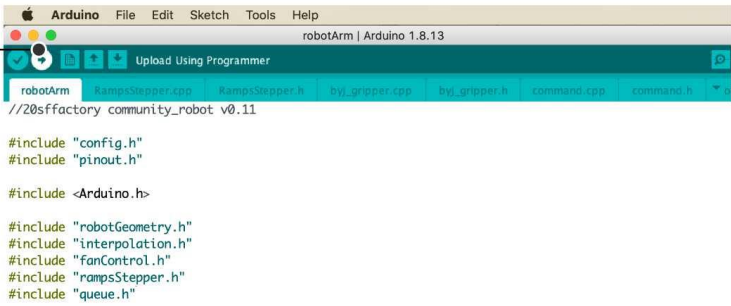
Select Board

Arduino Mega or Mega2560

Select Port

USB connected from PC to Board

Compile & Upload
See “Command Guide” for Controls



10 – Firmware

Command Guide (VAT)

VAT is a serial communication tool for Windows. It can be used to control robot arm.
link: sourceforge.net/projects/y-a-terminal (released under LGPLv2)

Open Terminal > Settings to configure terminal

Terminal Type: Text
Port Type: Serial COM Port
Serial Port: COM5 - USB-SERIAL CH340
Baud Rate: 115200
Data Bits: 8
Parity: None
Stop Bits: 1
Flow Control: None
☒ When connected, detect disconnect by monitoring the port every 500 ms
☒ When disconnected, try to reopen the port every 2000 ms
☒ Open/Start the New Terminal

Select Serial Port & Baud Rate

115200

Create Predefined Commands

Monitor

INFO: ROBOT ONLINE
INFO: SEND G28 TO CALIBRATE

Send Text
Send File
Send File (F6)

Serial port COM5 - USB-SERIAL CH340 (115200, 8, None, 1, None) is open and connected. 0.00.000

Monitor Input & Output Messages

Send Individual Commands here

Send Text
Send File
Send File (F6)

Predefine Command Texts & Descriptions

Page 1

1. G28
2. G28
3. G28
4. G28
5. G28
6. G28
7. G28
8. G28
9. G28
10. G28
11. G28
12. G28

Multi-line Text
G28 G28
G28 G28 G28

Multi-Line Text to define Continuous Moves

<Use with Caution>
Numerous commands may cause memory overflow in Arduino resulting in Missed Commands in Execution

Use Predefined Buttons to Move Robot

HOME
UNGRIP
GRIP
SEQUENCE1
SEQUENCE2

File with Command Sequence can be sent

<Use with Caution>
Numerous commands may cause memory overflow in Arduino resulting in Missed Commands in Execution